

ภาคผนวก ก

ตัวอย่างข้อโค้ดการทำงานหลักของโปรแกรม

1. ข้อโค้ดในส่วนของการลงทะเบียนไปยังซิฟเวิร์

```
void CSIPEngine::ExecuteL() {
    TInt status = iProfileRegistry->EnableL(*iProfile, *this);
    if (status == KErrNone) {
        // get the SIP connection used by the profile
        iConnection = iProfileRegistry->ConnectionL(*iProfile);
    } else
        // KErrPending
    {
        User::InfoPrint(_L("can't register"));
        iAppUi->RegisFail();
    }
}
```

เป็นส่วนของคลาสซิฟเอนจินที่จะทำการลงทะเบียนไปที่ซิฟเวิร์โดยใช้ฟังก์ชัน EnableL ของคลาส ProfileRegistry ฟังก์ชันนี้จะรีเทิร์นค่าออกมาว่ามีการผิดพลาดหรือไม่ หากไม่มีการผิดพลาดจะส่งค่าการเชื่อมต่อต่างๆ ไปไว้ในคลาส CSIPConnection แต่ถ้าหากเกิดความผิดพลาดจะไปเรียกคลาสที่เป็นส่วนติดต่อผู้ใช้ให้แสดงผลการลงทะเบียนไม่สำเร็จ

2. ข้อโค้ดในส่วนของการยืนยันการลงทะเบียน

```
void CSIPEngine::Subscribe(const TDesC8& user) {
    if (iSIPSubscribeDialogAssoc != NULL)
        delete iSIPSubscribeDialogAssoc ;
    iSIPSubscribeDialogAssoc = CreateSIPSubscribeDialogAssoc(user);
    if (iSIPClientTransaction != NULL)
        delete iSIPClientTransaction;
    iSIPClientTransaction = iSIPSubscribeDialogAssoc->SendSubscribeL();
}
```

เป็นส่วนของคลาสซิฟเอนจินที่จะทำการยืนยันการลงทะเบียนไปที่ซิฟเวิร์โดยใช้ฟังก์ชัน SendSubscribeL ของคลาส CSIPSubscribeDialogAssoc ฟังก์ชันนี้จะรีเทิร์นค่าออกมาเป็นค่าของทรานแซกชัน โดยก่อนที่จะส่งจะต้องสร้าง SIPSubscribeDialog ก่อนโดยใช้ฟังก์ชัน

CreateSIPSubscribeDialogAssoc โดยที่ฟังก์ชันนี้จะเป็นการกำหนดค่าต่างๆของ SIPSubscribeDialog เช่น ส่งจากผู้ใช้นี้คนหนึ่ง ส่งถึงผู้ใช้นี้คนหนึ่ง เป็นต้น

3. ขอโค้ดในส่วนของการ Invite

```
void CSIPEngine::Invite (const TDesC8& user) {
    if (iSIPInviteDialogAssoc != NULL)
        delete iSIPInviteDialogAssoc ;
    iSIPInviteDialogAssoc = CreateSIPInviteDialogAssoc(user);
    if (iSIPClientTransaction != NULL)
        delete iSIPClientTransaction;
    iSIPClientTransaction = iSIPInviteDialogAssoc->SendInviteL();
}
```

เป็นส่วนของคลาสซีพเอนจินที่จะทำการยืนยันการลงทะเบียนไปที่เซิร์ฟเวอร์โดยใช้ฟังก์ชัน SendInviteL ของคลาส CSIPInviteDialogAssoc ฟังก์ชันนี้จะรีเทิร์นค่าออกมาเป็นค่าของทรานแซกชัน โดยก่อนที่จะส่งจะต้องสร้าง SIPInviteDialog ก่อนโดยใช้ฟังก์ชัน CreateSIPInviteDialogAssoc โดยที่ฟังก์ชันนี้จะเป็นการกำหนดค่าต่างๆของ SIPInviteDialog เช่น ส่งจากผู้ใช้นี้คนหนึ่ง ส่งถึงผู้ใช้นี้คนหนึ่ง เป็นต้น

4. ขอโค้ดในส่วนของการบอกสถานะ

```
void CSIPEngine::Notify(const TDesC8& user) {
    CSIPRequestElements* iRequest = CreateSIPRequestMessage(user);
    if (iSIPClientTransaction != NULL)
        delete iSIPClientTransaction;
    iSIPClientTransaction = iConnection->SendRequestL(iRequest);
}
```

เป็นส่วนของคลาสซีพเอนจินที่จะทำการส่งสถานะต่างๆ ไปให้ที่เซิร์ฟเวอร์เพื่อส่งต่อไปให้กับผู้ที่ร้องขอมา โดยการสร้าง SIPRequestMessage ขึ้นมาโดยใส่รายละเอียดต่างๆ ด้วยฟังก์ชัน CreateSIPRequestMessage และส่งออกไปโดยใช้ฟังก์ชัน SendRequestL ของคลาส CSIPConnection

5. ขอโค้ดในส่วนของการดักจับการเปลี่ยนสถานะ

```
void CSIPEngine::ProfileRegistrationStatusChanged(TUint32 aSIPProfileId) {
    User::InfoPrint(_L("ProfileRegistrationStatusChanged"));
    if (aSIPProfileId == iProfile->Id()) {
        // Check that registration to the server was successful
    }
}
```

```

        if ( iProfile->Status() == CSIPProfile::ERegistered) {
            iConnection = iProfileRegistry->ConnectionL(*iProfile);
            iAppUi->ToSubScribe();
            iAppUi->ChangeContextStatus(0);
            iAppUi->SetTitleUsername();
            iAppUi->SetIsRegis(ETrue);
            //iAppUi->ChangeTitleUsername(GetOwnUsername());
            //iAppUi->SetTitleUsername();//Get From FileStore
        }
    }
}

```

เป็นส่วนของคลาสซีพเอนจิน ใช้สำหรับดักจับการเปลี่ยนแปลงสถานะ เพื่อทำการกำหนดค่าต่างๆ

6. ขอสกัดในส่วนของการดักจับการเข้ามาของ SIP Response

```

void CSIPEngine::IncomingResponse ( CSIPClientTransaction& aTransaction,
CSIPDialogAssocBase& /*aDialogAssoc*/) {
    if (aTransaction.ResponseElements()->CSeqHeader()->Method() == _L8("SUBSCRIBE")) {
        if (aTransaction.ResponseElements()->StatusCode() == 200){
            //...
        } else {
            //...
        }
    } else if (aTransaction.ResponseElements()->CSeqHeader()->Method() == _L8("INVITE")) {
        User::InfoPrint(_L("IncomingResponse"));
        // ACK is sent to any 2xx response
        // ...
    }
}

```

เป็นส่วนของคลาสซีพเอนจินที่จะทำการดักจับ SIP Response message ที่รับมา แล้วตรวจสอบว่าเป็น message ชนิดใดแล้วทำการประมวลผลว่าถ้าได้ message นี้มาจะทำอะไรต่อไป

7. ขอสไลด์ในส่วนของการดักจับการเข้ามาของ SIP Request (1)

```
void CSIPEngine::IncomingRequest (CSIPServerTransaction* aTransaction)
{
    User::InfoPrint(_L("IncomingRequest"));
    if (aTransaction->Type() == CSIPTransactionBase::EINVITE)
    {
        // ...
    }
    if (aTransaction->Type() == CSIPTransactionBase::ESUBSCRIBE) {
        // ...
    }
    if (aTransaction->Type() == CSIPTransactionBase::EMESSAGE) {
        // ...
    }
    delete aTransaction;
}
```

เป็นส่วนของคลาสซีพเอนจินที่จะทำการดักจับ SIP Request message ที่รับมา แล้วตรวจสอบว่าเป็น message ชนิดใดแล้วทำการประมวลผลว่าถ้าได้ message นี้มาจะทำอะไรต่อไป

8. ขอสไลด์ในส่วนของการดักจับการเข้ามาของ SIP Request (2)

```
void CSIPEngine::IncomingRequest ( CSIPServerTransaction* aTransaction , CSIPDialog& aDialog) {
    if (aTransaction->Type() == CSIPTransactionBase::EACK)
    {
        if (aDialog.IsAssociated(*iSIPInviteDialogAssoc))
        {
            //INVITE succesful
            User::InfoPrint(_L("IncomingACK"));
            // Start Session
            // ...
        }
    }
    if (aTransaction->Type() == CSIPTransactionBase::EBYE)
    {
        //Buddy left
    }
}
```

```

        User::InfoPrint(_L("IncomingBYE"));
        iServSocketsEngine->Disconnect();
    }
    delete aTransaction;
}

```

เป็นส่วนของคลาสซีพเอนจินที่จะทำการดักจับ SIP Request message ที่รับมา แล้วตรวจสอบว่าเป็น message ชนิดใดแล้วทำการประมวลผลว่าถ้าได้ message นี้มาจะทำอะไรต่อไป ในแบบที่สองนี้จะแตกต่างจากแบบแรก (ข้อ 7) ตรงที่แบบที่สองนี้จะป็น request ที่ส่งมาโดยไม่ผ่าน SIP Server ซึ่งเป็นกรณีที่เกิดขึ้นเมื่อมีการเปิดเซสชันกันระหว่างโทรศัพท์สองเครื่อง